Study-Mate Chat History

User: Insertion sort

Assistant: Algorithm for Insertion Sort: ``` Algorithm InsertionSort (a, n) { // sort the array a[1:n] into non decreasing order. for i := 2 to n do { key=a[i]; // Insert a[i] into the sorted part of the array, i.e., a [1: i -1] j=i-1; while (j>0 and a[j]>key) do { a[j+1]=a[j]; // move a[j] to its next position in the right side j=j-1; } a[j+1]=key; } } ``` **Performance Analysis:** Insertion sort takes linear time in the best case and quadratic time in the worst case. The complexity of insertion sort is generally stated as O(n^2), as O(n^2) covers linear time as well.

User: Big O notation

Assistant: Big O notation (O) is used to describe the limiting behavior of a function, particularly for classifying algorithms by how they respond to changes in input size. It provides an upper bound on the time complexity of an algorithm. **Definition:** Let f(n) and g(n) be functions mapping non-negative integers to non-negative real numbers. We say that f(n) is O(g(n)) if there exists a real constant c > 0 and an integer constant n0 > 0 such that f(n) ≤ cg(n) for all n ≥ n0. **Purpose:** The Big O notation is useful when we only have an upper bound on the time complexity of an algorithm, often by considering the worst-case time complexity. For sufficiently large values of n, multiplicative constants and lower-order terms of an exact running time are dominated by the effects of the input size itself and can be dropped for simplification. **Examples:** * 3n+2 = O(n) * 10n^2+4n+2 = O(n^2) * 6*2^n+n^2 = O(2^n) * 2100 = O(1) (O(1) denotes constant computing time) **Theorem:** If f(n) = a_m n^m + ... + a_1 n + a_0, then f(n) = O(n^m). **Efficiency Classes Relationship (for sufficiently large n):** Constant < log n < n < n log n < n^2 < n^3 < 2^n < n!